

Breve descrição do Firmware: Captura de Sons no ESP32-C6 com FreeRTOS

Sensores e Sinais

2025-2026

Paulo Pedreiras, Pedro Fonseca, Luis Silva (DETI/UA/IT)

Maio/2026

Este documento descreve a estrutura funcional do ficheiro `micMain.c`, desenvolvido para o **ESP32-C6**. O objetivo do código é capturar sinais sonoros oriundos de um microfone e processá-los concorrentemente e em tempo-real utilizando o modo **ADC Contínuo**. Este método permite amostrar o sinal a frequências elevadas (20.000 vezes por segundo ou superior) sem interromper a execução de outras partes do programa.

1 Breve Introdução ao FreeRTOS

Este código corre num microcontrolador, sendo necessário executar várias tarefas concorrentemente. O **FreeRTOS** é o kernel Tempo-Real (RTOS) que gere este fluxo através de dois conceitos principais:

- **Tarefas (Multitasking):** Uma tarefa funciona como um mini-programa independente. Neste código, existem duas: a `app_main` (configuração inicial e interação com a ADC) e a `pv_processor_task` (processa amostras de áudio). O FreeRTOS aloca, de forma transparente para o utilizador, o CPU a cada de uma delas de acordo com o seu estado de prontidão e prioridade, permitindo a sua execução concorrente.
- **Filas ou FIFOs (Queues):** Uma *Queue* é um mecanismo de comunicação entre tarefas, que permite o envio de dados na forma *First-In, First-Out*. Permite que uma tarefa envie dados para outra de forma segura, evitando que ambas tentem aceder à mesma memória simultaneamente. Garante também a sincronização entre as tarefas acima descritas. A `pv_processor_task` torna-se automaticamente activa (i.e. “acorda”) quando existem dados na *Queue*, ou seja, sempre que uma nova amostra de som fica disponível.

2 Funcionamento do Código

Após uma etapa inicial de configuração, o programa segue um fluxo cíclico com três etapas: **Captura** → **Conversão** → **Processamento**.

2.1 Configuração (Inicialização)

A função `app_main` configura o hardware do ADC para “ouvir” o som capturado pelo microfone (sinal analógico num pino específico da ADC) a **20 kHz**. Cria também a fila (**XQ**) que guardará as amostras de som e lança a tarefa de processamento (**Processor Task**).

2.2 Loop de Captura - Aquisição

Após a etapa de configuração, o `app_main` contém um ciclo principal que aguarda uma notificação do hardware:

1. **Interrupções:** Quando o ADC recolhe um bloco de dados (512 bytes), dispara uma função de *callback* (`s_conv_done_cb`) que dá “sinal verde” à tarefa principal.
2. **Conversão (Parsing):** A tarefa principal “acorda”, lê os sinais convertidos “em bruto” e converte-os em números decimais (*floats*) que representam a onda sonora.
3. **Preenchimento do Buffer:** O sistema continua a acumular estes blocos até ter um *buffer* de som completo com 2.048 amostras.

2.3 Entrega de Dados

Assim que o *buffer* está cheio, a tarefa principal utiliza a função `xQueueSend()` para colocar todo o bloco de áudio na Queue. De seguida, volta imediatamente ao processo de recolha do próximo bloco de amostras de áudio.

2.4 Tarefa de Processamento - Detecção de Tons

A tarefa `pv_processor_task` corre num ciclo infinito e utiliza a função `xQueueReceive()`, que diz ao FreeRTOS: “Acorda-me apenas quando houver um bloco de amostras de som na fila”.

- Quando os dados chegam, a tarefa “acorda” (passa ao estado Ready, sendo elegível para escalonamento) e imprime as primeiras 100 amostras no terminal para que se possa visualizar as amostras de som.
- Numa aplicação real, é aqui que se aplica o tratamento dos sinais sonoros. Poderiam ser usados e.g. filtro Passa-Banda ou FFTs (*Fast Fourier Transform*) para detectar frequências específicas. No caso deste projecto serão usados filtros Passa-Banda do tipo FIR.
- É também aqui que será implementada a máquina de estados com a lógica da aplicação.

3 Resumo do Fluxo de Dados

Etapa	Ação	Componente
Entrada	Conversão som para tensão	Hardware (Microfone)
Captura	Conversão da tensão para bits	<code>continuous_adc_init</code>
Transferência	Envio de blocos para um <i>buffer</i>	<code>s_conv_done_cb</code>
Entrega	Envio do <i>buffer</i> cheio para a Fila	<code>xQueueSend</code>
Saída	Leitura da Fila e impressão	<code>pv_processor_task</code>

Tabela 1: “Caminho” percorrido pelo sinal de áudio.